1. Software Design Specification

HYBRID_APP_ADA

Document Control	
Author(s)	Michael Gardner
Version	2.0.1
Status	Released
Status Date	2025-12-11
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	application
Library Role	N/A
Application Role	cli
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server
Execution Environment	linux, macos, windows
Processor Architecture	amd64, arm64
Deployment	native

Table of Contents

2. Introduction

2.1. Purpose

This Software Design Specification (SDS) describes the internal architecture, package structure, key type definitions, data-flow design, build strategy, and design decisions for **HYBRID_APP_ADA**.

2.2. Scope

This document covers:

- the 5-layer application profile of the shared hybrid architecture family,
- package hierarchy and dependency rules,
- key type definitions and contracts,
- static dependency injection via Ada generics,
- exception-boundary enforcement,
- build and deployment choices, and
- SPARK-readiness guidance.

2.3. References

- Software Requirements Specification (SRS).
- `docs/guides/architecture_enforcement.md`.
- Supporting UML diagrams maintained in `docs/diagrams/`.
- Ada 2022 Reference Manual.
- SPARK Reference Manual.

3. Architectural Overview

3.1. Architecture Style

HYBRID_APP_ADA uses the shared hybrid architecture family. Like the library profile, it preserves the same inner core: **Domain → Application → Infrastructure**. The application profile differs only at the outer boundary, where it uses **Presentation + Bootstrap** instead of the library API pattern.

Benefits of this style include:

- clear separation of concerns,
- testable business logic,
- swappable infrastructure adapters,
- compiler-enforced boundaries, and
- educational value for demonstrating correct use of the architecture.

3.2. Layer Architecture

Layer / Boundary	Purpose	Depends On
Application	Use cases, commands, and port definitions.	Domain
Bootstrap	Composition root that wires the application together through static dependency injection.	All layers
Domain	Pure business concepts, error types, and value objects.	Nothing
Infrastructure	Driven adapters implementing technical concerns and external integration.	Application + Domain
Presentation	Driving adapters and user-interface entry points.	Application only

Detailed architecture guidance is maintained in `docs/guides/`. Supporting UML diagrams are maintained in `docs/diagrams/`.

3.3. Layer Responsibilities

Layer	Responsibility
Application	Use-case orchestration, command DTOs, port definitions, and output formatting.
Bootstrap	Static composition root that performs generic instantiation and binds ports to adapters.
Domain	Pure business logic, value objects, error types, and result handling with zero external dependencies.
Infrastructure	Adapter implementations, exception-to-result conversion, and interaction with the outside world.
Presentation	User-interface handling, argument parsing, and error formatting through the application boundary.
Version package	Cross-cutting version metadata kept outside the hexagonal layers.

3.4. Dependency Rules

Component	May Depend On
Application	Domain only.
Bootstrap	All layers; Bootstrap is the intentional composition root.
Domain	Nothing (zero dependencies).
Infrastructure	Application and Domain.
Presentation	Application only; Presentation shall not depend directly on Domain.

Architecture conformance is enforced through layered controls:
- project/build configuration constraints where applicable,
- active validation using `scripts/arch_guard.py`,
- execution of `scripts/arch_guard.py` during normal developer build workflow, and
- execution of `scripts/arch_guard.py` in CI/CD before non-conforming changes are accepted.

4. Package Structure

4.1. Directory Layout

```
src/
├── hybrid_app_ada.ads              # Root package
├── version/
│   ├── hybrid_app_ada.ads          # Parent package spec
│   └── hybrid_app_ada-version.ads  # Version constants
│
├── domain/
│   ├── domain.ads
│   ├── domain-unit.ads
│   ├── error/
│   │   ├── domain-error.ads
│   │   └── domain-error-result.ads
│   └── value_object/
│       ├── domain-value_object.ads
```

```
│       ├── domain-value_object-option.ads
│       └── domain-value_object-person.ads
│
├── application/
│   ├── application.ads
│   ├── error/
│   │   └── application-error.ads
│   ├── command/
│   │   ├── application-command.ads
│   │   └── application-command-greet.ads
│   ├── port/
│   │   ├── application-port.ads
│   │   ├── inbound/
│   │   │   └── application-port-inbound.ads
│   │   └── outbound/
│   │       ├── application-port-outbound.ads
│   │       └── application-port-outbound-writer.ads
│   └── usecase/
│       ├── application-usecase.ads
│       └── application-usecase-greet.ads
│
├── infrastructure/
│   ├── infrastructure.ads
│   └── adapter/
│       ├── infrastructure-adapter.ads
│       └── infrastructure-adapter-console_writer.ads
│
├── presentation/
│   ├── presentation.ads
│   └── adapter/
│       ├── presentation-adapter.ads
│       └── cli/
│           ├── presentation-adapter-cli.ads
│           └── command/
│               ├── presentation-adapter-cli-command.ads
│               └── presentation-adapter-cli-command-greet.ads
│
└── bootstrap/
    ├── bootstrap.ads
    └── cli/
        ├── bootstrap-cli.ads
        └── bootstrap-cli-run.adb
```

4.2. Package Descriptions

4.2.1. Domain Layer

Package	Purpose	SPARK
`Domain`	Layer root.	On
`Domain.Error`	Error type with kind and bounded message.	On
`Domain.Error.Result`	Generic Result[T] monad.	On
`Domain.Unit`	Unit type for void-like operations.	On
`Domain.Value_Object`	Value object root.	On
`Domain.Value_Object.Option`	Option[T] monad.	On

Package	Purpose	SPARK
`Domain.Value_Object.Person`	Person value object.	On

4.2.2. Application Layer

Package	Purpose	SPARK
`Application`	Layer root.	On
`Application.Command.Greet`	Greeting command DTO.	On
`Application.Error`	Re-exports Domain.Error for Presentation consumption.	On
`Application.Port.Outbound.Writer`	Writer outbound port definition.	On
`Application.Usecase.Greet`	Greeting use case.	On

4.2.3. Infrastructure Layer

Package	Purpose	SPARK
`Infrastructure`	Layer root.	Off
`Infrastructure.Adapter.Console_Writer`	Console output adapter implementing the Writer port.	Off

4.2.4. Presentation and Bootstrap

Package	Purpose	SPARK
`Bootstrap.CLI.Run`	Static composition root that wires adapters, ports, and use cases.	Off
`Hybrid_App_Ada.Version`	Version information for CLI display and release metadata.	Off
`Presentation.Adapter.CLI.Command.Greet`	CLI command adapter that parses arguments and maps results to exit codes.	Off

5. Type Definitions

5.1. Domain Types

5.1.1. Error_Kind

```
type Error_Kind is (Validation_Error, IO_Error);
```

5.1.2. Error_Type

```
type Error_Type is record
   Kind    : Error_Kind;
   Message : Error_Strings.Bounded_String;
end record;
```

5.1.3. Result (Generic)

```
generic
   type T is private;
package Generic_Result is
   type Result (Is_Success : Boolean) is private;
```

```
   function Ok (Value : T) return Result;
   function Error (Kind : Error_Kind; Message : String) return Result;

   function Is_Ok (Self : Result) return Boolean;
   function Value (Self : Result) return T;
   function Error_Info (Self : Result) return Error_Type;
end Generic_Result;
```

5.1.4. Person Value Object

```
Max_Name_Length : constant := 100;
package Person_Strings is new Ada.Strings.Bounded.Generic_Bounded_Length
  (Max => Max_Name_Length);

type Person is record
   Name_Value : Person_Strings.Bounded_String;
end record;

function Create (Name : String) return Person_Result.Result;
function Get_Name (Self : Person) return String;
function Is_Valid_Person (P : Person) return Boolean;
```

Design decisions:

- public record required for generic instantiation,
- validation only through `Create`,
- immutable value semantics,
- bounded strings avoid heap allocation.

5.2. Application Types

5.2.1. Greet_Command DTO

```
Max_DTO_Name_Length : constant := 256;
package Greet_Strings is new
  Ada.Strings.Bounded.Generic_Bounded_Length (Max => Max_DTO_Name_Length);

type Greet_Command is record
   Name : Greet_Strings.Bounded_String;
end record;
```

This boundary intentionally allows a larger DTO size than the Domain validation limit, so the Domain may evolve independently while maintaining defense in depth.

5.2.2. Application.Error Re-Export Pattern

```
with Domain.Error;

package Application.Error is
   subtype Error_Type is Domain.Error.Error_Type;
   subtype Error_Kind is Domain.Error.Error_Kind;
   package Error_Strings renames Domain.Error.Error_Strings;

   Validation_Error : constant Error_Kind := Domain.Error.Validation_Error;
   IO_Error         : constant Error_Kind := Domain.Error.IO_Error;
end Application.Error;
```

This preserves the Presentation → Application boundary while still exposing the domain-owned error model through a zero-overhead re-export.

6. Component Design

6.1. Domain Layer Design

The Domain layer owns value objects, error types, the generic result monad, and pure validation logic. It is designed for SPARK compatibility and zero external dependencies.

6.2. Application Layer Design

6.2.1. Greeting Use Case

```
generic
   with function Writer (Message : String) return Unit_Result.Result;
package Application.Usecase.Greet is
   function Execute (Cmd : Greet_Command) return Unit_Result.Result;
end Application.Usecase.Greet;
```

`Execute` validates the command via the Domain, formats the greeting through `Format_Greeting`, and delegates output through the outbound Writer port.

6.2.2. Format_Greeting Placement

A deliberate design decision places greeting formatting in the Application layer, not the Domain. The Domain provides data (`Get_Name`); the Application decides how that data is rendered for use by the application boundary.

6.3. Infrastructure Layer Design

`Infrastructure.Adapter.Console_Writer` performs console I/O, catches exceptions through `Functional.Try.Try_To_Result_With_Param`, maps them to domain errors, and converts `Functional.Result` values back into `Domain.Error.Result` values before returning to the rest of the system.

6.4. Presentation Layer Design

`Presentation.Adapter.CLI.Command.Greet` receives a use-case function as a generic formal, parses arguments, invokes the application boundary, formats errors using `Application.Error`, and returns an exit code.

6.5. Bootstrap Design

`Bootstrap.CLI.Run` acts as the composition root. It:

- wires Infrastructure to the outbound Writer port,
- wires the use case to the port,
- wires the CLI command adapter to the use case, and
- executes the command.

All dependency injection is static and resolved at compile time.

7. Design Patterns

Pattern	Use in HYBRID_APP_ADA
Application Service Re-Export	Application.Error re-exports Domain.Error to preserve boundaries.
DTO Boundary Isolation	Commands allow larger bounds than domain validation to support independent evolution.
Hexagonal Architecture	Ports defined in Application; adapters in Infrastructure and Presentation.

Pattern	Use in HYBRID_APP_ADA
Railway-Oriented Programming	Result-monad based error handling using `Domain.Error.Result.Generic_Result`.
Static Dependency Injection	Ada generics wire ports to implementations in Bootstrap.
Value Object Pattern	Immutable, validated domain primitives such as `Person`.

7.1. Exception Boundary Specification

This project uses a layered exception-boundary strategy consistent with the shared hybrid architecture family.

Layer	Functional.Try	Manual `exception`	Rationale
Application	N/A	Forbidden	Use-case orchestration through Result types only.
Bootstrap	Optional	Allowed	Startup and wiring may fail fast.
Domain	N/A	Forbidden	Core logic and value semantics; runtime exceptions indicate bugs.
Infrastructure	Required	Forbidden	Boundary to external systems and I/O.
Main / entry point	Optional	Allowed	Top-level clean exit handling.
Presentation	Required	Forbidden	Boundary to user input and UI concerns.
Test	Optional	Allowed	Tests may validate exceptional scenarios.

Required pattern:

- Presentation and Infrastructure boundaries shall use `Functional.Try` wrappers.
- Manual exception handlers are prohibited in Presentation and Infrastructure.
- Domain and Application shall not contain `exception` handling logic.

Core philosophy:

- if an exception occurs in Domain or Application, it indicates a bug, missing validation, or incomplete contracts,
- such failures should be corrected, not normalized into routine runtime handling.

Validation enforcement:

- `scripts/arch_guard.py` validates direct dependency rules as part of local developer workflow and CI/CD,
- project/build configuration provides complementary boundary enforcement where applicable.

8. Data Flow

8.1. Success Path

```
User CLI input
  -> main / Bootstrap.CLI.Run
  -> Presentation.Adapter.CLI.Command.Greet
  -> Application.Usecase.Greet.Execute
  -> Domain.Value_Object.Person.Create
  -> Application.Format_Greeting
```

```
  -> Infrastructure.Adapter.Console_Writer.Write
  -> exit code 0
```

8.2. Error Path

```
Invalid user input
  -> Domain validation fails
  -> Domain.Error.Result propagated upward
  -> Presentation formats Application.Error for the user
  -> exit code 1
```

9. Concurrency Design

9.1. Thread Safety

- Domain: stateless and pure; inherently thread-safe.
- Application: stateless orchestration; thread-safe.
- Infrastructure: adapters may carry state in future variants.
- Presentation: current CLI design is single-threaded.

9.2. Future Concurrency Support

Potential future extensions may use Ada tasks and protected objects, including Ravenscar-oriented designs for stricter targets. The port abstraction keeps this path open without changing the core architecture.

10. Performance and Memory Design

10.1. Zero-Overhead Abstractions

- static dispatch through generics,
- stack-based discriminated Result values,
- bounded strings in the Domain,
- pure functions that are friendly to optimization.

10.2. Memory Management

- no heap allocation in the Domain,
- stack allocation for result values and commands,
- deterministic cleanup without garbage collection.

11. Security Design

11.1. Input Validation

All input validation is enforced through the Domain layer, with the command DTO boundary providing defense in depth.

11.2. Error Information

Error messages are structured and bounded. They are intended to be safe for user display and should not contain sensitive information.

12. Build and Deployment

12.1. Project Structure

```
hybrid_app_ada/
├── hybrid_app_ada.gpr
```

```
├── alire.toml
├── Makefile
└── src/
```

Design decision:

- single-project structure rather than an aggregate, to keep deployment simple, Alire-compatible, and fast to build.

12.2. Platform Support

Platform	Status	Notes
Linux	Full	Fully supported.
macOS	Full	Primary development platform.
Windows	Full	Supported through CI and runtime validation.
BSD	Manual	Builds may succeed but are not CI-covered.
Embedded	Planned	Would require custom adapters.

13. SPARK Readiness Assessment

13.1. Overview

The project is designed with future SPARK verification in mind. The architecture follows SPARK-compatible patterns where practical, even though the full application is not a pure SPARK system.

13.2. Layer Assessment

Layer	SPARK Readiness	Notes
Application	Medium	Mostly pure orchestration and generic formals.
Bootstrap	Medium	Static wiring, but instantiates non-SPARK layers.
Domain	High	Pure functions, bounded types, no side effects.
Infrastructure	Low	Uses Text_IO and boundary exception handling.
Presentation	Low	Uses command-line and UI concerns.

13.3. SPARK-Compatible Components

Recommended SPARK targets include:

- `Domain.Value_Object.Person`,
- `Domain.Value_Object.Option.Generic_Option`,
- `Domain.Error.Result.Generic_Result`,
- `Domain.Error.Error_Type`, and
- `Application.Command.Greet` plus much of `Application.Usecase.Greet`.

13.4. Future SPARK Integration

Future SPARK work would add explicit `SPARK_Mode` aspects, contracts on public functions, explicit `SPARK_Mode => Off` markers for non-SPARK layers, and GNATprove runs over the Domain-first core.

14. Testing Strategy

14.1. Test Organization

```
test/
├── unit/         # Domain + Application logic
```

```
├── integration/  # Cross-layer tests
├── e2e/          # Full system tests
└── common/       # Test framework
```

Current metrics:

- 85 unit tests,
- 16 integration tests,
- 8 end-to-end tests,
- 109 total tests.

14.2. Testing Approach

- Unit tests focus on Domain and Application logic.
- Integration tests exercise cross-layer collaboration.
- End-to-end tests validate the application through the CLI boundary.

15. Version 2.0.0 Enhancements

15.1. Result Combinator Patterns

Version 2.0.0 adopted the richer combinator set from `functional` 3.0.0, including:

- `Bimap`,
- `Ensure`,
- `With_Context`,
- `Fallback`,
- `Recover`, and
- `Tap`.

These improve expressiveness without changing the underlying Result-oriented design.

15.2. Windows CI Support

Version 2.0.0 also formalized Windows CI/runtime support as part of the platform matrix.

16. Appendices

16.1. Change History

Version	Date	Author	Changes
2.0.1	2025-12-11	Michael Gardner	Added Section 6.1 Exception Boundary Specification with required/prohibited patterns and layered enforcement rules.
2.0.0	2025-12-08	Michael Gardner	Added Result combinators section; Windows CI support; updated test metrics to 109 total tests.
1.0.0	2025-11-27	Michael Gardner	Initial release.